DeepAgents 기반 워크플로우

Status	
Date	@March 21, 2026 12:45 AM

LlamaIndex 기반에서 DeepAgents로의 전환

기존 시스템의 문제점

1. 컨텍스트 관리의 한계

- 에이전트가 참고해야 하는 모든 내용을 시스템 프롬프트에 직접 주입하는 구조
- 기본 시스템 프롬프트가 이미 긴 상태에서 chat history와 tool output이 누적되면서 전체 컨텍스트가 과도하게 비대해짐
- 메모리가 token_limit 초과 시 오래된 메시지를 단순 삭제하는 방식이라, 대화가 길어질수록 초반 컨텍스트가 유실됨

2. 에이전트 간 흐름 제어의 한계

- LlamaIndex의 AgentWorkflow는 실행할 때마다(매 턴마다) Context가 초기화되는 구조여서, 현재 라우팅된 에이전트 정보 등을 유지하기 위해 세션 내내 보관되는 별도의 컨텍스트 저장 클래스를 직접 만들어야 했음
- handoff를 통해 이뤄지는 에이전트 간 제어권이 수평으로 이전됨
- 위의 구조때문에 멀티턴 대화 시 매 턴마다 현재 에이전트를 수동으로 메인 에이전트로 리셋해야 했음 (한번 넘어간 제어권이 자동으로 돌아오지 않는 경우가 많음)
- 에이전트 체인이 길어질수록 흐름 추적과 결과 취합이 어려워짐

→ 전반적으로 프롬프트로 에이전트를 제어하려는 시도 + 프레임워크의 내부로직을 수정하거나 커스텀하여 사용하는 빈도 증가

DeepAgents

DeepAgents의 미들웨어와 기본 제공 도구들

미들웨어	사용 훅	역할
FilesystemMiddleware	`wrap_model_call`, `wrap_tool_call`	파일시스템 도구 제공 (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `execute`). 시스템 프롬프트에 파일시스템 지침 주입. 대형 도구 결과를 파일로 eviction
MemoryMiddleware	`before_agent`, `wrap_model_call`	`AGENTS.md` 파일에서 에이전트 메모리 로드 → 시스템 프롬프트에 주입
SkillsMiddleware	`before_agent`, `wrap_model_call`	`SKILL.md` 프론트매터에서 스킬 메타데이터 로드 → 시스템 프롬프트에 스킬 문서 주입
SubAgentMiddleware	`wrap_model_call`	`task` 도구 제공 (서브에이전트로 위임). 사전 컴파일된 서브에이전트 스펙으로 도구 빌드
SummarizationMiddleware	`wrap_model_call`	토큰 사용량 초과 시 대화 자동 압축. 오래된 메시지를 백엔드에 오프로드
SummarizationToolMiddleware	`wrap_model_call`	`compact_conversation` 도구 제공 (수동 압축). SummarizationMiddleware와 조합하여 사용
PatchToolCallsMiddleware	`before_agent`	응답에 `tool_call_id`가 누락된 경우 패치

미들웨어 훅(Hook) 종류

AgentMiddleware는 **2가지 스타일**, 총 **6개의 훅**을 제공

Node-style hooks (순차 실행)

Hook	시점	반환값	용도
`before_agent`	에이전트 시작 전	`dict` (state update) 또는 `None`	초기 상태 로드, 사전 검증
`before_model`	매 모델 호출 전	`dict` 또는 `None`	메시지 제한 체크, 로깅, `jump_to` 로 분기
`after_model`	매 모델 응답 후	`dict` 또는 `None`	응답 로깅, 상태 카운터 업데이트
`after_agent`	에이전트 완료 후	`dict` 또는 `None`	정리 작업, 최종 로깅

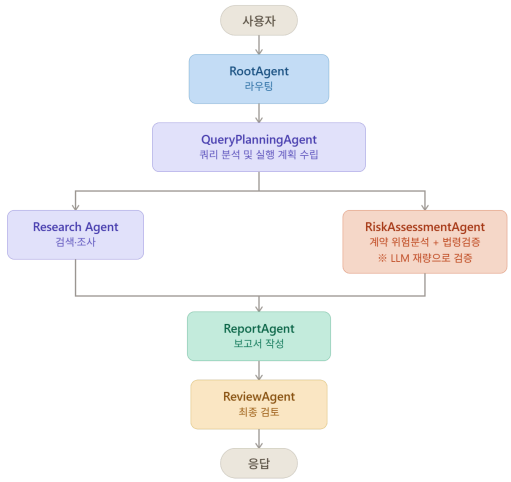
Wrap-style hooks (호출을 감싸서 제어)

Hook	대상	반환값	용도
`wrap_model_call`	모델 호출	`ModelResponse` 또는 `ExtendedModelResponse`	재시도, 캐싱, 응답 변환/검증
`wrap_tool_call`	도구 호출	`ToolMessage` 또는 `Command`	도구 결과 변환, 대형 결과 eviction

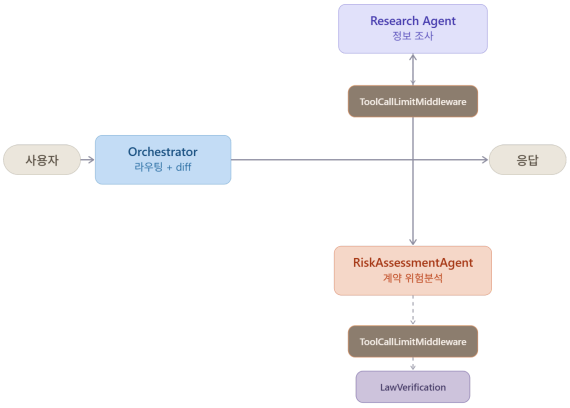
> 모든 hook에 비동기 버전 존재 (`awrap_model_call`, `abefore_agent`)

기존 구조와 현재 구조의 시나리오 비교 (한국주택도시공사 기준)

Llamaindex 워크플로우



Deepagents기반 워크플로우



기존 문제에 대응하여 적용된 내용

주요 내용

1. **컨텍스트 관리 → MemoryMiddleware + SummarizationMiddleware**
   - **AGENTS.md 기반 프로젝트 메모리**: 에이전트 초기화 시 AGENTS.md의 위치와 사용 지침이 프롬프트에 추가되며, 실제 내용은 read_file 도구로 필요 시 동적으로 참조함. 시스템 프롬프트에 전체 내용을 하드코딩하지 않으므로 컨텍스트 부담이 줄어듦.
   - **자동 오프로딩**: 모든 tool 결과와 history가 20k 토큰이 초과될 때 자동으로 파일 저장, 경로+미리보기만 prompt에 유지
   - **자동 요약**
     - 긴 대화에서 token 제한 도달 시 자동으로 과거 메시지를 요약해 state에 영구 저장
     - 최근 메시지는 유지하며, 앞으로의 턴에서도 요약된 히스토리를 사용

3. **흐름 제어 → 서브에이전트 오케스트레이션과 Hook의 사용**
   - 메인 에이전트가 `task` 도구로 서브에이전트에 작업을 위임하되, 결과를 자동 수렴하여 흐름이 명시적임
   - 각 서브에이전트는 격리된 컨텍스트에서 실행되므로, 서로 다른 하위 작업의 대화 히스토리가 오염되지 않음
   - Hook을 통해 반드시 도구를 사용해야 하는 경우 기존 미들웨어나 커스텀 미들웨어를 만들어 적용
     - 적용 예시 : '전세조항 위험도 에이전트'의 응답에 대해 법령에 대한 검증이 반드시 필요한 상황

       → after_model 훅을 이용해 메인 에이전트로 리턴하기 전 법령 검증을 실행

3. **툴 호출 제어 → ToolCallLimitMiddleware**
   - 서브에이전트가 tool call을 반복하면 context가 빠르게 소진되는데, 기존에는 Tool Call 호출 정도를 프롬프트로만 제한하고 있었음. (n번 이상 호출 금지)
   - ToolCallLimitMiddleware 사용 시 Tool call 제한 초과 시 해당 tool 호출을 차단하고 에이전트는 그 상태의 메시지 히스토리를 기반으로 최종 응답을 생성

4. **자동 툴 결과 요약 → FilesystemMiddleware + SummarizationToolMiddleware**
   - 메시지 히스토리와 별개로 에이전트가 긴 tool 출력을 요약할 수 있도록 함

5. **Planning 과정 지원**

- 내장 write_todos 도구로 복잡한 작업을 분해하고 순차적으로 진행하며 진행 상황 또한 추적함(pending/in_progress/completed)
- 장기간에 걸친 멀티스텝 작업에 특히 유용하며 기존 워크플로우처럼 별도의 planning agent를 두지 않아도 됨

개발한 커스텀 미들웨어	사용 훅	역할
TracingMiddleware	`before_model`, `after_model`	서브에이전트 LLM 호출 시작/종료 로깅, 소요 시간 측정
LawVerificationMiddleware	`awrap_model_call`	**사후 검증** — final response에서 법령 인용 감지 → RAG 검증 → LLM 보정 → 수정된 응답 반환
ForceToolUseMiddleware	`awrap_model_call`	전세조항 위험도 에이전트의 도구 사용 강제

멀티 턴 응답 비교 테스트

```
[질문리스트]
```

```
1. 전세계약서를 작성하는데 이런 특약이 있는데 검토해주세요.
"임차인은 입주 전 파손된 부분과  노후화된 시설 모두 교체/수리해야한다."

2. 내가 계약만기가 다가오는데 집주인이 잠적했어. 어떻게 해야해?

3. 전세보증에 가입이 거절되는 경우도 있어?

4. 전세계약을 하러갈건데 임대인없이 부동산이 대리로 계약을 할 경우에 어떤 위험이있어?

5. 법인 임차인의 전세보증보험 신청 방법을 알려줘

6. 집주인이 갑자기 바뀌었어. 내용증명을 보내고싶은데 어떻게 해야해?
```

Q. 마지막 턴까지 왔을 때 응답 결과(동일 모델 사용)

▼ **llamaindex 기반 워크플로우 멀티턴 결과** → 후반 질문들을 할 때 mcp 도구를 사용하지 못하는 에러가 발생하여 에이전트가 자체적으로 대답하기 시작

핵심 답변

전세 계약이 만료되었는데 집주인이 연락이 되지 않을 경우, 임차인은 **보증금 반환 청구**와 **퇴거 절차**를 순차적으로 진행해야 하며, 필요 시 **법원에 소송**을 제기하거나 **주거안정 지원**을 신청할 수 있습니다.

상세 분석 및 설명

- **보증금 반환 청구 절차**

1. **내용증명 발송** – 계약 종료일로부터 1개월 이내에 집주인에게 보증금 반환을 요구하는 내용증명을 등기우편으로 발송합니다.
2. **보증금 반환 청구서 제출** – 내용증명 발송 후 2주 이내에 관할 **주택임대차보호법 담당 부서(주거복지센터·주민센터 등)**에 보증금 반환 청구서를 제출합니다.
3. **조정·중재 요청** – 지방자치단체 조정위원회에 조정을 신청해 집주인과 합의를 시도합니다.

- **퇴거 절차**

1. **퇴거 통보** – 계약 종료일에 맞춰 집주인에게 퇴거 의사를 서면(내용증명)으로 통보합니다.
2. **법원에 퇴거명령 신청** – 집주인이 퇴거를 거부하거나 연락이 두절된 경우, 관할 **지방법원**에 **퇴거명령 신청**을 할 수 있습니다.
3. **집행관에 의한 퇴거** – 법원 명령이 내려지면 집행관이 퇴거를 집행합니다.

- **보증금 반환 소송**
- **소송 제기 요건**: 계약서, 영수증·계약금 영수증, 내용증명 사본, 입주 전·후 사진·동영상 등 증거 자료.
- **제소 기간**: 보증금 반환 청구는 계약 종료 후 **2년** 이내에 제기해야 합니다(주택임대차보호법 제6조).
- **소송 절차**: 소액사건(보증금 ≤ 2천만원)인 경우 **소액재판**을 이용하면 간편하게 진행할 수 있습니다.
- **임차인 보호 제도**
- **주거안정 지원**: 지방자치단체에 따라 임차인에게 임시 거주 지원(공공임대주택·임시거처) 및 생활비 지원을 제공하는 경우가 있습니다.
- **주거복지센터 상담**: 보증금 반환 및 퇴거 문제에 대해 무료 법률 상담을 받을 수 있습니다.

추가 확인 및 조치 사항

1. **증거 확보**

- 입주 전·후 사진·동영상, 계약서, 영수증, 내용증명 사본 등을 모두 보관합니다.

1. **내용증명 발송**

- 내용증명은 등기우편으로 발송하고, 발송일과 수신인 서명을 확인합니다.

1. **주거복지센터·주민센터 방문**

- 보증금 반환 청구서와 관련 서류를 지참해 상담 및 조정 신청을 진행합니다.

1. **법원 소송 준비**

- 필요 서류(계약서, 영수증, 내용증명, 사진·동영상 등)를 정리하고, 변호사 상담을 통해 소송 전략을 수립합니다.

1. **임시거처 확보**

- 지방자치단체 주거안정 지원 프로그램을 확인하고, 임시거처 신청 절차를 진행합니다.

위 절차를 차례대로 진행하면, 집주인이 잠적한 상황에서도 보증금 반환과 퇴거를 법적으로 보호받을 수 있습니다. 필요 시 변호사와 상담하여 구체적인 소송 전략을 수립하시기 바랍니다.

▼ **deepagents 기반 워크플로우 멀티턴 결과** → 에러 없이 끝까지 응답이 완성됨

집주인(임대인) 변경 시 내용증명 발송 절차

(검색된 HUG·주택임대차보호법·민법 근거 포함)

1️⃣ 임대인(집주인) 변경에 대한 법적 기본 원칙

법령	조문	핵심 내용
주택임대차보호법 제3조(대항력 등)	"임차인이 주택을 인도받고 전입신고·주민등록을 하면, 그 다음 날부터 제3자(새 소유자 등)에게도 효력이 발생한다."	→ **전입·점유만 유지하면 새 소유자에게도 보증금 청구·우선변제권이 그대로 유지**.
민법 제618조(임대차의 승계)	"목적물의 소유권이 이전될 경우, 임대차는 승계된다. 다만, 임대인이 계약을 해지하지 아니하고 승계된 경우에 한한다."	→ **소유권 이전(매매·상속) 시 기존 전세계약은 자동 승계**(임차인이 해지 통보하지 않은 경우).
민법 제623조(임대인의 의무)	"임대인은 목적물을 인도하고 사용·수익에 필요한 상태를 유지할 의무가 있다."	→ **임대인 변경 후에도 보증금 반환·주택 인도 의무는 새 임대인에게 승계**.
민법 제635조(해지통고)	"임차인이 계약을 해지하고자 할 때는 1개월 전(건물·주택) 통고한다."	→ **임대인 변경과 무관하게 계약 종료·보증금 반환 청구는 통고 절차를 따라야 함**.

> ***핵심: 전입·점유가 유지되는 한 대항력·우선변제권은 새 임대인에게 그대로 이어진다. 하지만 보증금 반환·계약 종료 통보는 반드시 서면(내용증명)으로 해야 법적 효력이 확보됩니다.***

2️⃣ HUG(주택도시보증공사)·HF 가이드 – 임대인 변경 시 절차

단계	HUG·HF 가이드 내용	실무 적용 팁
① 임대인 변경 신고	"임대인 변경 시 매매계약서·소유권 이전 서류 또는 새 임대인 인적사항을 제출해야 함." (문서: **33. (보증발급) 임대인 변경 등 조건변경.md**)	- **매매계약서·등기부등본**을 스캔·첨부 - 새 임대인 연락처·주소를 정확히 기재
② 보증조건 변경 신청	"전세보증금 반환보증 가입자는 '임대인 변경' 신청을 해야 보증 효력 유지." (전세안심 온라인 클래스)	보증보험 가입자라면 **보증조건변경(임대인변경) 신청**을 즉시 진행
③ 이행청구·내용증명 활용	"보증금 반환 청구 시 '임대차 계약 종료 증빙'(내용증명·문자·통화녹취·공시송달 등) 제출 가능." (보증이행_이행청구_제출서류_RAG최적화.md)	내용증명 외에도 **문자·카톡 캡처·통화녹취**를 함께 보관하면 증거력 강화
④ 대항력 유지	"전입·점유를 유지하면 새 소유자에게도 대항력·우선변제권이 지속." (전세안심 온라인 클래스)	전입신고·주민등록 사본을 **내용증명에 첨부**하면 임대인 변경 후에도 권리 주장에 유리

3️⃣ 내용증명에 반드시 포함할 항목

구분	구체적 내용 (예시)
수신인	새 임대인(성명·주소·연락처)
발신인	임차인(성명·주민등록번호·연락처)
계약 기본 정보	전세계약일, 보증금액, 계약기간, 전입일, 전입신고·주민등록 사본
요청 내용	1) 보증금 전액 반환(계좌명·번호 명시) 2) 계약 종료(전입·점유 유지) 확인 및 인도일 지정
법적 근거	- 주택임대차보호법 제3조(대항력) - 민법 제623조(임대인 의무) - 민법 제635조(해지통고)

구분	구체적 내용 (예시)
회신·이행 기한	발송일 기준 **10일 이내** 회신·보증금 반환 요청
추가 조치 안내	회신·보증금 반환이 없을 경우 **주택임대차 분쟁조정위원회·소액사건심판·보증이행 청구**를 진행한다는 내용
첨부 서류	- 전세계약서 사본 - 등기부등본(새 임대인) - 전입신고·주민등록 사본 - 보증금 영수증·이체 내역 - (보증보험 가입 시) 보증증권 사본

4️⃣ 내용증명 발송 실무 흐름

```
flowchart TD
    A[임대인(새 집주인) 확인] --> B[필요 서류(등기부, 계약서, 전입신고) 확보]
    B --> C[내용증명 초안 작성(위 표 내용 반영)]
    C --> D[우체국 방문 → 내용증명 발송(원본 2부 + 사본 1부)]
    D --> E[우편물 접수 영수증·우편번호 보관]
    E --> F[회신·보증금 반환 확인]
    F -->|미회신| G[분쟁조정위원회·소액심판·보증이행 청구 진행]
```

5️⃣ 구체적인 서식 예시 (1페이지)

```
[제목] 전세계약 보증금 반환·임대차 종료 통보 내용증명

수신 : ○○○(새 임대인) 귀하
발신 : □□□(임차인) 귀하

1. 계약 개요
   - 전세계약일 : 2022-03-01
   - 보증금 : 1,200만원
   - 계약기간 : 2022-03-01 ~ 2024-02-28
   - 전입일 : 2022-03-02 (전입신고·주민등록 사본 첨부)

2. 요청 사항
   가. 위 계약에 따라 보증금 전액(1,200만원)을 아래 계좌로 반환해 주시기 바랍니다.
      → 계좌 : 국민은행 123-456-789012 (예금주: ○○○)
   나. 계약 종료일(2024-02-28)까지 주택 인도 및 전입신고 확인을 완료해 주시기 바랍니다.

3. 법적 근거
   - 주택임대차보호법 제3조(대항력)
   - 민법 제623조(임대인의 의무)
   - 민법 제635조(해지통고)

4. 회신 기한
   - 본 내용증명 발송일(2024-03-20)로부터 10일 이내(2024-03-30) 회신 및 보증금 반환을 요청합니다.

5. 향후 조치
   - 기한 내 회신·보증금 반환이 없을 경우, 주택임대차 분쟁조정위원회 및 소액사건심판에 신청할 예정임을 알려드립니다.

첨부 : 전세계약서 사본, 등기부등본(새 임대인), 전입신고·주민등록 사본, 보증금 영수증

2024-03-20
임차인 □□□ (서명)
```

6️⃣ 임대인 변경 후 추가 고려 사항

상황	권고 조치
보증보험(전세보증금 반환보증) 가입자	HUG·HF에 **'임대인 변경' 신청**을 즉시 진행 → 보증조건(임대인) 변경이 반영돼야 보증금 반환 청구가 원활함.
임대인(전 소유자)와 새 임대인 모두에게 내용증명 발송	기존 임대인에게도 "보증금 반환·계약 승계"를 통보해 **채무 승계 증거**를 확보.
전입·점유 유지	전입신고·주민등록을 유지하면 **대항력·우선변제권**이 새 소유자에게 그대로 이어짐. 전입을 포기하면 권리 상실 위험.
공시송달 필요 시	새 임대인 주소가 불명확하거나 회신이 없을 경우, **우체국 공시송달**(관보·관보지에 송달) 신청 → "통지·청구가 이루어졌다"는 법적 효력 확보.
보증금 반환 청구 소송	회신·보증금 반환이 없을 경우, **보증이행 청구**(HUG) → **지급명령·소액사건심판** 순서로 진행.

📌 요약

1. **임대인 변경** → 등기부·연락처 확인 → 새 임대인에게 **내용증명** 발송.
2. 내용증명에 **계약 정보·요청·법적 근거·기한·첨부 서류**를 모두 명시.
3. **우체국에서 내용증명 발송** → 영수증·우편번호 보관.
4. **회신·보증금 반환이 없으면** → 주택임대차 분쟁조정위원회 → 소액사건심판 → 보증이행 청구·지급명령 등 단계적 진행.
5. **보증보험 가입자**는 반드시 **임대인 변경 신청**을 하고, **전입·점유 유지**를 통해 대항력을 보전한다.

위 절차대로 진행하면 **법적 효력**을 갖춘 통지·청구가 완료됩니다. 추가로 **공시송달 신청 방법**이나 **보증이행 청구 서류**가 필요하면 알려 주세요. 바로 상세히 안내해 드리겠습니다.

Deep Agents vs LangGraph: 아키텍처 비교

두 프레임워크는 근본적으로 다른 오케스트레이션 패턴을 사용

Deep Agents: 메인 에이전트 + 서브에이전트 패턴

```
┌──────────────────────────────────────────┐
│  StateGraph (최소한의 래퍼)               │
│                                          │
│  START ──→ orchestrator ──→ END          │
│              │                           │
│    ┌─────────┴──────────┐                │
│    │  Main Agent        │                │
│    │  (미들웨어 스택)     │                │
│    │                    │                │
│    │  task() 도구 호출 ──→├─→ Sub-Agent 1 (research)      │
│    │                    ├─→ Sub-Agent 2 (risk-assess)   │
│    │                    ├─→ Sub-Agent N (동적 생성)      │
│    └────────────────────┘                │
└──────────────────────────────────────────┘
```

- 단일 orchestrator 노드만 존재 — 그래프는 `START → orchestrator → END` 뿐
- 서브에이전트는 `task()` 도구 호출로 동적 생성/소멸 (그래프 엣지가 아님)

- 기능은 미들웨어 스택으로 주입 (Todo, Memory, Skills, Filesystem, Summarization 등)
- 서브에이전트는 상태 격리 — 부모 상태의 일부만 전달받고 결과를 `ToolMessage`로 반환
- LLM이 런타임에 어떤 서브에이전트를 호출할지 결정

LangGraph: 노드-엣지 그래프 패턴

```
┌──────────────────────────────────────┐
│  StateGraph (명시적 노드/엣지)          │
│                                      │
│  START ──► retrieve ──► grade_docs   │
│                          │      │    │
│                     (relevant) (not) │
│                          │      │    │
│                          ▼      ▼    │
│                      generate  web_search │
│                          │      │    │
│                          ▼      │    │
│                       check ◄────┘    │
│                        │  │           │
│                   (ok) │  │ (retry)   │
│                        ▼  └──► generate │
│                       END            │
└──────────────────────────────────────┘
```

- 여러 노드가 명시적 엣지로 연결 — `add_node()`, `add_edge()`, `add_conditional_edges()`
- 각 노드는 순수 함수 `(State) → Partial<State>`
- 공유 State를 통해 노드 간 데이터 전달
- 조건부 엣지 함수가 다음 노드를 결정

두 프레임워크 중 DeepAgents를 적용한 이유는

1. 명확한 관리포인트

- Deep Agents는 그래프 `START → orchestrator → END` 하나뿐
- 서브에이전트는 독립된 스펙(이름, 프롬프트, 도구)으로 정의되니 각각 개별 관리 가능
- LangGraph 순수 방식은 노드·엣지가 늘어날수록 그래프 자체가 복잡해짐

2. 작게 시작할 수 있음

- orchestrator + 서브에이전트 1~2개로 바로 동작
- 새 기능 추가 시 서브에이전트 스펙만 추가하면 됨

3. LangGraph로 확장 가능

- Deep Agents 자체가 LangGraph `CompiledStateGraph`를 반환하므로, 더 큰 LangGraph 그래프의 노드로 편입 가능
  - 지금의 orchestrator를 하나의 노드로 두고, 앞뒤에 전처리/후처리 노드를 붙이거나 조건부 분기를 추가하는 식

▼ (기록용) deepagents에서 에이전트 응답에 대한 법령 검증을 반드시 하게하는 방법

1. **risk-assess-agent를 subgraph로 구성해서 메인 에이전트에 붙이기 (복잡도가 높고 소요시간이 길어 오버된 설계)**
   - main → risk-subgraph (risk-agent → verification-agent)
2. **middleware only (after_model 훅 사용+ jump_to 옵션)**
   - 모델 응답 (UI 표출) → regex 로 법령이 언급됨을 감지 → HumanMessage 주입 → jump_to model 옵션 (모델 호출로 다시 이동) → 에이전트가 search_hug_docs 호출 → 검증 응답 (UI 2번째 표출) → 종료
   - 한계: 응답이 2회 표출됨 (미검증 + 검증)
3. **middleware only (wrap_model_call)**
   a. **wrap_model_call 훅 사용 (시스템 프롬프트 주입)**
      - 매 모델 호출 시 시스템 프롬프트에 "법령 인용 시 반드시 search_hug_docs로 검증하세요" 주입 → 에이전트가 자체적으로 도구 호출하여 검증
      - 한계 : LLM 재량에 의존하여 지침 무시 가능
   b. **wrap_model_call (사후 검증) → 채택**
      - final response 확인 → 미들웨어가 직접 RAG + 검증 LLM 호출 → 응답 수정 후 반환
      - 한계 : 스트리밍 환경에서 원본 응답이 먼저 UI에 표출된 후 수정 응답으로 교체되어 보일 수 있으나 현재로선 이게 최선임

방법	검증 보장	UI 응답 횟수	스트리밍 호환	구현 복잡도
1. 서브그래프	O	1회	O (서브그래프 내부만 비노출)	높음
2. after_model + jump_to	O	2회	O	낮음
3-1. wrap (프롬프트 주입)	X (LLM 재량)	1회	O	낮음
3-2. wrap (사후 검증)	O	1회 (단, 일시 중복)	부분적	중간